Lab 3

Name: Mahdi Davoodzadeh
SN: 405725536
Repository: https://github.com/deep-observability-lab/OS-Tracing.git

1. Objective

In this final lab, I investigated an unknown compiled program by observing its output and tracing its kernel activity. I first ran the program and made a prediction about its behavior. Next, I used `perf` and LTTng to identify the important system call and recorded scheduling events to understand the behavior of its threads. Finally, I used the **Uaddr vs Waiter** view in Trace Compass to examine the waiting intervals and explain the apparent deadlock.

I performed the main investigation with an **8-second run** and collected a separate **30-second recording** to display the persistent waiting behavior over a longer interval.

2. Test Program and Initial Prediction

I downloaded the compiled program from the Lab 3 source directory of the assignment repository and copied it to `src/deadlock_m`. It is a 64-bit, dynamically linked Linux ELF executable. The file name `deadlock_m` suggested that the program might demonstrate a deadlock involving mutexes.

I ran the program first, inspected its output, and used tracing to test this prediction. The experiments used Ubuntu 24.04.5 LTS, kernel `6.17.0-40-generic`, `perf` 6.17.13, LTTng 2.13, Babeltrace, and Trace Compass.

3. Initial 8-Second Execution

I used `scripts/02_run_baseline.sh` to run the program with an 8-second timeout. I used unbuffered output so that the messages would still be saved if the program stopped making progress.

The command used by the script was equivalent to:

```
timeout --signal=TERM --kill-after=2s 8s \
  stdbuf -o0 -e0 ./src/deadlock_m
```

The process reached the timeout and exited with **status 124**. Its standard error was empty. The saved standard output contained **66 lines**, preserved in `logs/baseline/02_program.stdout`. The complete output appears in Appendix A; the excerpts below highlight the initial activity and final waits.

The first ten lines show the threads acquiring and releasing locks:

```
Thread 1: Waiting for A
Thread 1: Locked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 2: Unlocked B
Thread 2: Waiting for A
Thread 1: Unlocked A
Thread 1: Waiting for B
Thread 1: Locked B
Thread 2: Locked A
```

The final twelve lines show the transition to the final waits:

```
Thread 1: Unlocked A
Thread 1: Waiting for B
Thread 1: Locked B
Thread 2: Locked A
Thread 1: Unlocked B
Thread 1: Waiting for A
Thread 2: Unlocked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 1: Locked A
Thread 2: Waiting for A
Thread 1: Waiting for B
```

At first, both threads repeatedly acquired and released locks `A` and `B`. The output ended with Thread 2 waiting for `A` and Thread 1 waiting for `B`.

From this observation, I predicted a **circular-wait deadlock**: Thread 1 was likely holding `A` while waiting for `B`, and Thread 2 was likely holding `B` while waiting for `A`. I tested this prediction by examining futex operations, scheduling events, and the waiting intervals in Trace Compass.

4. Identifying the Key System Call with Perf

Because the program appeared to use two mutexes, I suspected the Linux **`futex`** system call. I checked the executable's imported symbols using `nm -D` and found `pthread_create`, `pthread_join`, `pthread_mutex_init`, `pthread_mutex_lock`, and `pthread_mutex_unlock`. These symbols supported my hypothesis that the program used POSIX threads and mutexes.

I then used `scripts/03_perf_futex_stat.sh` to measure `futex` and scheduling events during a separate, 6-second controlled run. The selected Perf tracepoints were:

```
syscalls:sys_enter_futex
syscalls:sys_exit_futex
sched:sched_switch
sched:sched_wakeup
sched:sched_waking
```

The recorded counters were:

Perf event	Count
syscalls:sys_enter_futex	25
syscalls:sys_exit_futex	25
sched:sched_switch	34
sched:sched_wakeup	0
sched:sched_waking	49

The program again reached the timeout, with exit status **124**. The nonzero `futex` counts showed that the mutex hypothesis was consistent with actual kernel activity. I also recorded scheduling events, as required by the assignment.

These counters cover the measured command and its descendants, including the launcher and timeout process. I used LTTng and Trace Compass to associate the synchronization activity with specific futex addresses and worker threads.

5. LTTng Recording of the 8-Second Run

I used `scripts/04_lttng_futex_trace.sh` to record the `futex` system call with LTTng and enabled the following scheduling tracepoints:

```
sched_switch
sched_wakeup
sched_waking
sched_process_fork
sched_process_exit
```

In particular, I enabled the system call through LTTng's syscall interface:

```
sudo lttng enable-event --kernel --syscall futex
```

I ran the program with an 8-second timeout while tracing was active. After the run, I stopped and destroyed the LTTng session, retaining the recorded data and a user-readable copy for Trace Compass at:

```
logs/lttng/04_futex_trace_gui/kernel
```

The Babeltrace export contained **35,580 events**, including the following counts:

LTTng event	Recorded count
`syscall_entry_futex`	4,858
`syscall_exit_futex`	4,856
`sched_switch`	12,862
`sched_wakeup`	6,443
`sched_waking`	6,444
`sched_process_fork`	60
`sched_process_exit`	57

These **system-wide counts** include `deadlock_m` and background processes. I identified the application's waiting threads through the scheduler records and the address-specific Trace Compass view.

I used the `sched_process_fork` records to identify the process and its workers in this run:

Role	PID / TID
Main process	112956
First worker TID	112957
Second worker TID	112958

The scheduling records also showed `deadlock_m` threads being scheduled out and later woken up during the working part of the run. Near the end, their recorded waiting behavior changed.

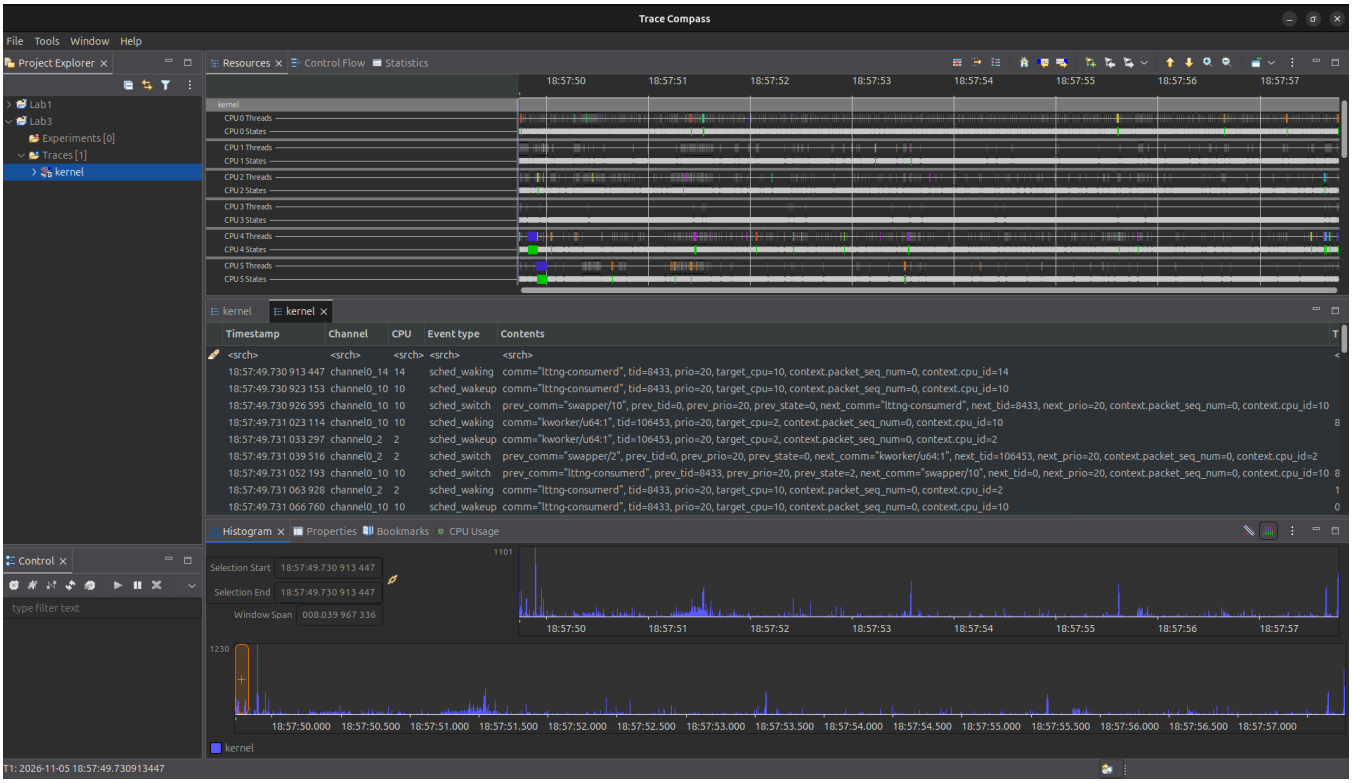


Figure 1 — Scheduling events in the 8-second trace. Trace Compass displays the Resources timeline and recorded scheduler events over the approximately eight-second capture.

6. Eight-Second Trace Compass Analysis — Uaddr vs Waiter

I opened the LTTng trace in Trace Compass and selected:

```
Lab3 → Traces → kernel → Views
     → Futex Contention Analysis → Uaddr vs Waiter
```

I zoomed out to view the entire recording and located worker TIDs `112957` and `112958`. In this view, each parent row represents a futex user-space address (`uaddr`), and its child rows represent the threads observed waiting on it. The gray bars show the waiting intervals along the timeline.

The important rows in my 8-second trace were:

Futex address (`uaddr`)	**Waiter TID**	**Observation**
`94419365879872`	`112957`	Several short waits earlier in the run
`94419365879872`	`112958`	Repeated short waits followed by a long final wait
`94419365879936`	`112957`	A long final wait

In the earlier part of the trace, I saw short, repeated waiting bars, consistent with the threads successfully competing for and releasing mutexes. Near the end of the recording, **two longer bars overlapped on two different futex addresses**. Both continued until the program was terminated by the timeout.

I also examined the saved LTTng futex events using `scripts/06_analyze_futex_addresses.sh`:

Futex address	**`op = 128`**	**`op = 129`**
`94419365879872`	11	10
`94419365879936`	1	0

For these calls, `op = 128` is `FUTEX_WAIT_PRIVATE` and `op = 129` is `FUTEX_WAKE_PRIVATE`. The private flag applies to synchronization between threads in the same process. A wait checks the expected futex value and sleeps when it matches; a wake requests that waiting threads become runnable. The first address had repeated waits and wakes during normal activity. The second had a recorded wait with no subsequent wake event on that address in the observed interval.

The final matching wait entries included:

```
18:57:56.460628761  uaddr=94419365879872  op=128  val=2
18:57:56.560944830  uaddr=94419365879936  op=128  val=2
```

The Trace Compass view associated the final long wait on the first address with TID `112958`, and the final long wait on the second address with TID `112957`. These simultaneous waits, together with the program's last messages about locks `A` and `B`, strongly supported my prediction of a **circular-wait mutex deadlock**.

I used the application's lock messages to infer the relationship between locks `A` and `B`, and the trace to identify the worker TIDs and their waiting addresses. The final intervals ended with timeout-driven process termination.

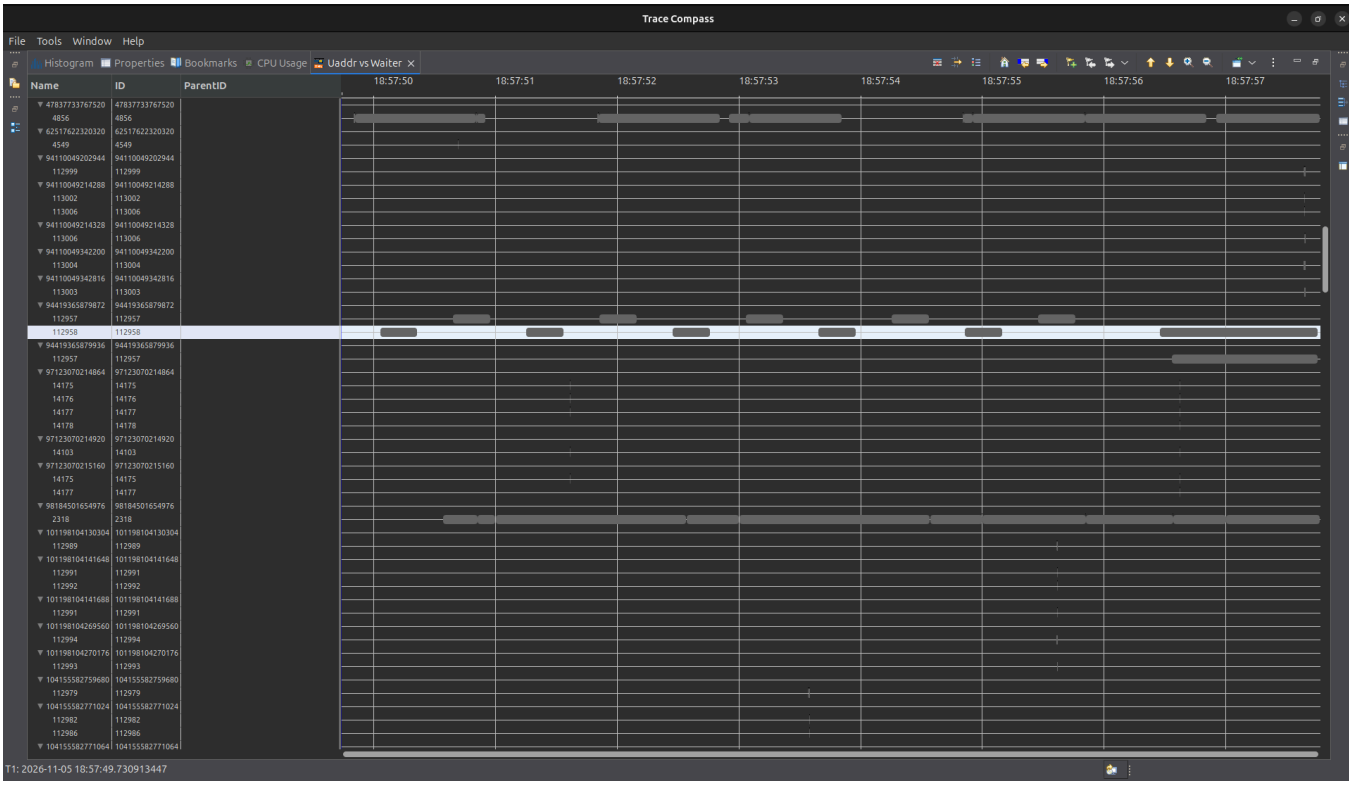


Figure 2 — Uaddr vs Waiter for the 8-second recording. At address 94419365879872, TID 112958 changes from short waits to a long final wait. TID 112957 waits on address 94419365879936 during the overlapping final interval.

7. The 30-Second Recording

I recorded a longer run to show the persistent, overlapping waits more clearly. This extended the visualization of the behavior identified in the 8-second investigation.

I used `scripts/07_lttng_30s_trace.sh` to record the same futex and scheduling events for up to 30 seconds. This recording was kept separate from the earlier one:

```
logs/lttng/07_futex_30s_trace_gui/kernel
```

The recording completed successfully, and the program again reached the timeout (**exit status 124**).

7.1 Program Output from the 30-Second Run

I saved all 66 output lines in `logs/lttng/07_program.stdout` and reproduced them in Appendix B. Its **last 12 lines** were:

```
Thread 1: Unlocked A
Thread 1: Waiting for B
Thread 1: Locked B
Thread 2: Locked A
Thread 1: Unlocked B
Thread 1: Waiting for A
Thread 2: Unlocked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 1: Locked A
Thread 2: Waiting for A
Thread 1: Waiting for B
```

The output again ended with Thread 2 waiting for `A` and Thread 1 waiting for `B`, followed by timeout termination.

7.2 Uaddr vs Waiter During the 30-Second Run

I opened the new trace in Trace Compass and used its scheduling records to identify the new process and thread IDs:

Role	PID / TID
Main process	127567
First worker TID	127568
Second worker TID	127569

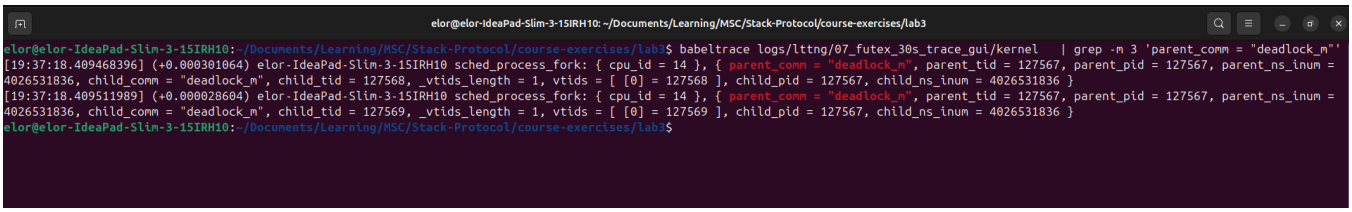


Figure 3 — Process and thread identification for the 30-second run. Both fork records have parent PID 127567; the worker TIDs are 127568 and 127569, with child PID 127567 showing their shared process.

The new run had different futex addresses, as expected for a separate execution:

Futex address (`uaddr`)	Waiter TID	Observation
95635432890432	127568	Short, repeated waits early in the trace
95635432890432	127569	Long final wait
95635432890496	127568	Long final wait

In the zoomed-out view, I observed two long and overlapping waiting bars from approximately **19:37:25 until 19:37:48**, or around **23 seconds**. These bars made the persistent blocking much easier to see than in the shorter trace. The final waiting intervals ended when the timeout terminated the process.

I analyzed the two recordings separately using each run’s own PID, TIDs, futex addresses, and timestamps.

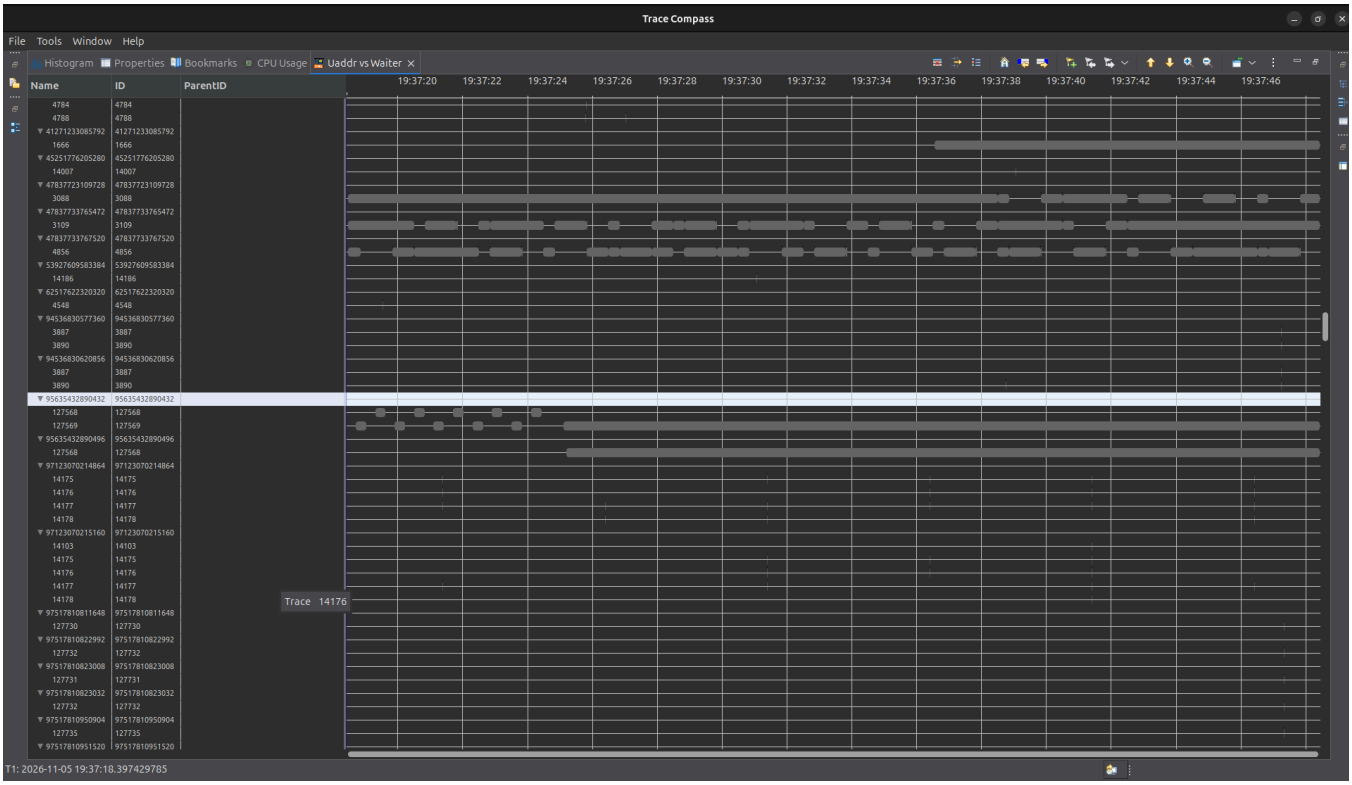


Figure 4 — Uaddr vs Waiter for the 30-second recording. TID 127569 waits on address 95635432890432 while TID 127568 waits on address 95635432890496. The two long bars overlap from approximately 19:37:25 until termination near 19:37:48.

8. Explanation of the Deadlock and the Two Visualizations

The key system call was **`futex`**, which provides kernel-assisted waiting and waking for thread synchronization. Uncontended mutex operations can complete in user space; contention can lead a thread to wait in the kernel on a futex word. This connects the program's mutex operations to the observed wait and wake calls. The imported `pthread_mutex_lock` and `pthread_mutex_unlock` functions identified the program as a mutex-based multithreaded application; the Perf and LTTng results then confirmed that the program exercised the kernel futex mechanism.

In the Uaddr vs Waiter view, a short gray bar represented a short waiting interval. The repeated short bars, wake operations, and lock/unlock messages together showed the threads making progress during the earlier part of the run. A long bar showed that a thread remained waiting on a futex. In both recordings, the two worker threads ended in **overlapping long waits on different futex addresses**.

Combined with the printed lock messages, this was strong evidence of a circular-wait deadlock:

- Thread 1 appeared to retain lock `A` while waiting for lock `B`.
- Thread 2 appeared to retain lock `B` while waiting for lock `A`.

Neither thread could obtain the lock it needed to continue, so the process remained blocked until `timeout` terminated it. The visualization provided evidence of persistent waiting; the program output provided the logical relationship between locks `A` and `B`.

The 8-second trace shows the transition from repeated short waits to simultaneous blocking. The 30-second trace shows that the final blocking persists for approximately 23 seconds, making the circular-wait interpretation clearer.

Appendix A: Complete 8-Second Output

The following reproduces all 66 lines from `logs/baseline/02_program.stdout`.

```
Thread 1: Waiting for A
Thread 1: Locked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 2: Unlocked B
Thread 2: Waiting for A
Thread 1: Unlocked A
Thread 1: Waiting for B
Thread 1: Locked B
Thread 2: Locked A
Thread 1: Unlocked B
Thread 1: Waiting for A
Thread 2: Unlocked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 1: Locked A
Thread 2: Unlocked B
Thread 2: Waiting for A
Thread 1: Unlocked A
Thread 1: Waiting for B
Thread 1: Locked B
Thread 2: Locked A
Thread 1: Unlocked B
Thread 1: Waiting for A
Thread 2: Unlocked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 1: Locked A
Thread 2: Unlocked B
Thread 2: Waiting for A
Thread 1: Unlocked A
Thread 1: Waiting for B
Thread 1: Locked B
Thread 2: Locked A
Thread 1: Unlocked B
Thread 1: Waiting for A
Thread 2: Unlocked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 1: Locked A
Thread 2: Unlocked B
Thread 2: Waiting for A
Thread 1: Unlocked A
Thread 1: Waiting for B
Thread 1: Locked B
Thread 2: Locked A
Thread 1: Unlocked B
Thread 1: Waiting for A
Thread 2: Unlocked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 1: Locked A
Thread 2: Unlocked B
Thread 2: Waiting for A
Thread 1: Unlocked A
Thread 1: Waiting for B
Thread 1: Locked B
Thread 2: Locked A
Thread 1: Unlocked B
Thread 1: Waiting for A
Thread 2: Unlocked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 1: Locked A
Thread 2: Unlocked B
Thread 2: Waiting for A
Thread 1: Unlocked A
Thread 1: Waiting for B
Thread 1: Locked B
Thread 2: Locked A
Thread 1: Unlocked B
Thread 1: Waiting for A
Thread 2: Unlocked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 1: Locked A
Thread 2: Waiting for A
Thread 1: Waiting for B
```

Appendix B: Complete 30-Second Output

The following reproduces all 66 lines from `logs/lttng/07_program.stdout`.

```
Thread 1: Waiting for A
Thread 1: Locked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 2: Unlocked B
Thread 2: Waiting for A
Thread 1: Unlocked A
Thread 1: Waiting for B
Thread 1: Locked B
Thread 2: Locked A
Thread 1: Unlocked B
Thread 1: Waiting for A
Thread 2: Unlocked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 1: Locked A
Thread 2: Unlocked B
Thread 2: Waiting for A
Thread 1: Unlocked A
Thread 1: Waiting for B
Thread 1: Locked B
Thread 2: Locked A
Thread 1: Unlocked B
Thread 1: Waiting for A
Thread 2: Unlocked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 1: Locked A
Thread 2: Unlocked B
Thread 2: Waiting for A
Thread 1: Unlocked A
Thread 1: Waiting for B
Thread 1: Locked B
Thread 2: Locked A
Thread 1: Unlocked B
Thread 1: Waiting for A
Thread 2: Unlocked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 1: Locked A
Thread 2: Unlocked B
Thread 2: Waiting for A
Thread 1: Unlocked A
Thread 1: Waiting for B
Thread 1: Locked B
Thread 2: Locked A
Thread 1: Unlocked B
Thread 1: Waiting for A
Thread 2: Unlocked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 1: Locked A
Thread 2: Unlocked B
Thread 2: Waiting for A
Thread 1: Unlocked A
Thread 1: Waiting for B
Thread 1: Locked B
Thread 2: Locked A
Thread 1: Unlocked B
Thread 1: Waiting for A
Thread 2: Unlocked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 1: Locked A
Thread 2: Unlocked B
Thread 2: Waiting for A
Thread 1: Unlocked A
Thread 1: Waiting for B
Thread 1: Locked B
Thread 2: Locked A
Thread 1: Unlocked B
Thread 1: Waiting for A
Thread 2: Unlocked A
Thread 2: Waiting for B
Thread 2: Locked B
Thread 1: Locked A
Thread 2: Waiting for A
Thread 1: Waiting for B
```